



UNIDADE III

Engenharia de Software Ágil

Prof. Me. Edson Moreno

Introdução à fusão do produto e do processo

- A fusão do produto de software com o processo promove a integração contínua entre o que está sendo construído (produto) e como está sendo construído (processo).
- Esta fusão de produto e processo permite desenvolver artefatos ou sistemas de software ajudando as empresas desenvolvedoras a consolidar suas operações reduzindo custos e aumentando a produtividade.
- A gestão ágil elimina a separação entre planejar e executar, criando um ciclo de melhoria contínua em que cada decisão no processo impacta na entrega de valor ao produto.
- Melhora a eficiência, flexibilidade e maior alinhamento com as necessidades reais do cliente.



Fonte: ClipArt.

Fusão do produto e do processo de software

- Esta fusão representa a integração estreita entre a definição do produto final e as atividades necessárias para desenvolvê-lo.
- É uma abordagem que assegura que os requisitos do produto software e seus processos de desenvolvimento sejam considerados simultaneamente.
 - O produto software é o resultado final do desenvolvimento, inclui elementos como código, interfaces do usuário e do hardware, documentação, funcionalidades e desempenho.
 - O processo de software abrange as camadas da engenharia de software como qualidade, processos, métodos, ferramentas e técnicas utilizadas para criar o produto software.

Neste capítulo serão abordados:

- Domínio da análise do engenheiro de software
- Joint Application Development (JAD)
- Gerenciamento da equipe de desenvolvimento

Domínio da análise do engenheiro de software

- O domínio de análise do engenheiro de software se refere ao conjunto de áreas de conhecimento e aspectos técnicos, organizacionais e de negócio que o engenheiro de software deve considerar.
- A análise envolve a compreensão detalhada do problema ou necessidades a serem implementadas, conhecimentos de normas e padrões da qualidade do software, processos organizacionais, de desenvolvimento e tecnologias envolvidas, ferramentas, infraestrutura de TI e comunicação com clientes e usuários.
- Estes conhecimentos são definidos na engenharia de software como uma tecnologia em camadas (Pressman, 2021): Foco na qualidade, Processo, Métodos e Ferramentas.

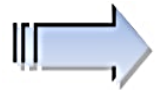
Camadas da engenharia
de software

Fonte: Pressman (2021).



Qualidade

- A qualidade estabelece regras, padrões, normas, procedimentos e atividades que orientam o desenvolvimento de software de alta qualidade.
- Leva à cultura de melhoria contínua dos processos e abordagens cada vez mais eficientes da engenharia de software.



Satisfação do usuário = produto adequado

+ máxima qualidade

+ entrega dentro do orçamento e do cronograma

Exemplos de Normas e Modelos da Qualidade:

- NBR ISO/IEC 25010: Requisitos de qualidade e avaliação de sistemas e software.
- ISO 12207 – Processos do Ciclo de Vida do Software.
- ISO 9001 – Sistema de Gestão da Qualidade.
- CMMI (Capability Maturity Model Integration).
- MPS.BR (Melhoria do Processo de Software Brasileiro).

Processo

Define a estrutura fundamental para a concepção e desenvolvimento de software, orientado pelos padrões e normas da qualidade estabelecida.

- O processo assegura a integração das camadas tecnológicas, permitindo um desenvolvimento de software racional e oportuno.
- Os modelos de processos do ciclo de desenvolvimento do software são apresentados basicamente em seus frameworks como: Elicitação, Análise, Especificação e Modelagem, Construção, Implementação, Implantação e Validação.

Principais frameworks de modelos de processos

- PSP – Processo de Software Pessoal
- TSP – Processo de Software da Equipe
- Cascata (Waterfall ou Sequencial Linear)
- Prototipagem
- Modelo Incremental
- Modelo RAD
- Modelo Espiral
- RUP – Processo Unificado Racional

Métodos

- É o conjunto estruturado de procedimentos, regras e técnicas utilizadas para guiar o desenvolvimento do software para alcançar uma meta, fim ou conhecimento.
- Um método assegura que as etapas do ciclo de vida do software sejam seguidas de maneira sistemática e eficiente.
- O método determina os passos a serem seguidos na construção do software ou parte dele.
- Envolve um amplo conjunto de tarefas, desde a gestão de equipes, análise de requisitos, projeto e construção de programas, testes, entrega, suporte e manutenção.

Exemplos de métodos:

- Metodologias ágeis – Coleção de metodologias baseadas na prática efetiva da construção de software.
- Matriz de responsabilidades – Método de gestão de equipes que define papéis e responsabilidades a cada membro da equipe em relação às atividades e tarefas do projeto.
- PERT/CPM – Técnica de Revisão e Controle de Programas / Método do Caminho Crítico.

Ferramentas

São recursos tecnológicos que dão apoio automatizado para os processos e métodos.

- As ferramentas de software agilizam as tarefas, aumentam a produtividade, comunicação e colaboração entre as equipes.
- Dentre essas ferramentas, temos as de gestão de projetos, modelagem, ambiente de desenvolvimento integrado (IDE), controle de versões, inspeção da qualidade do código, testes, documentação e diversas outras.

Exemplos de ferramentas de software

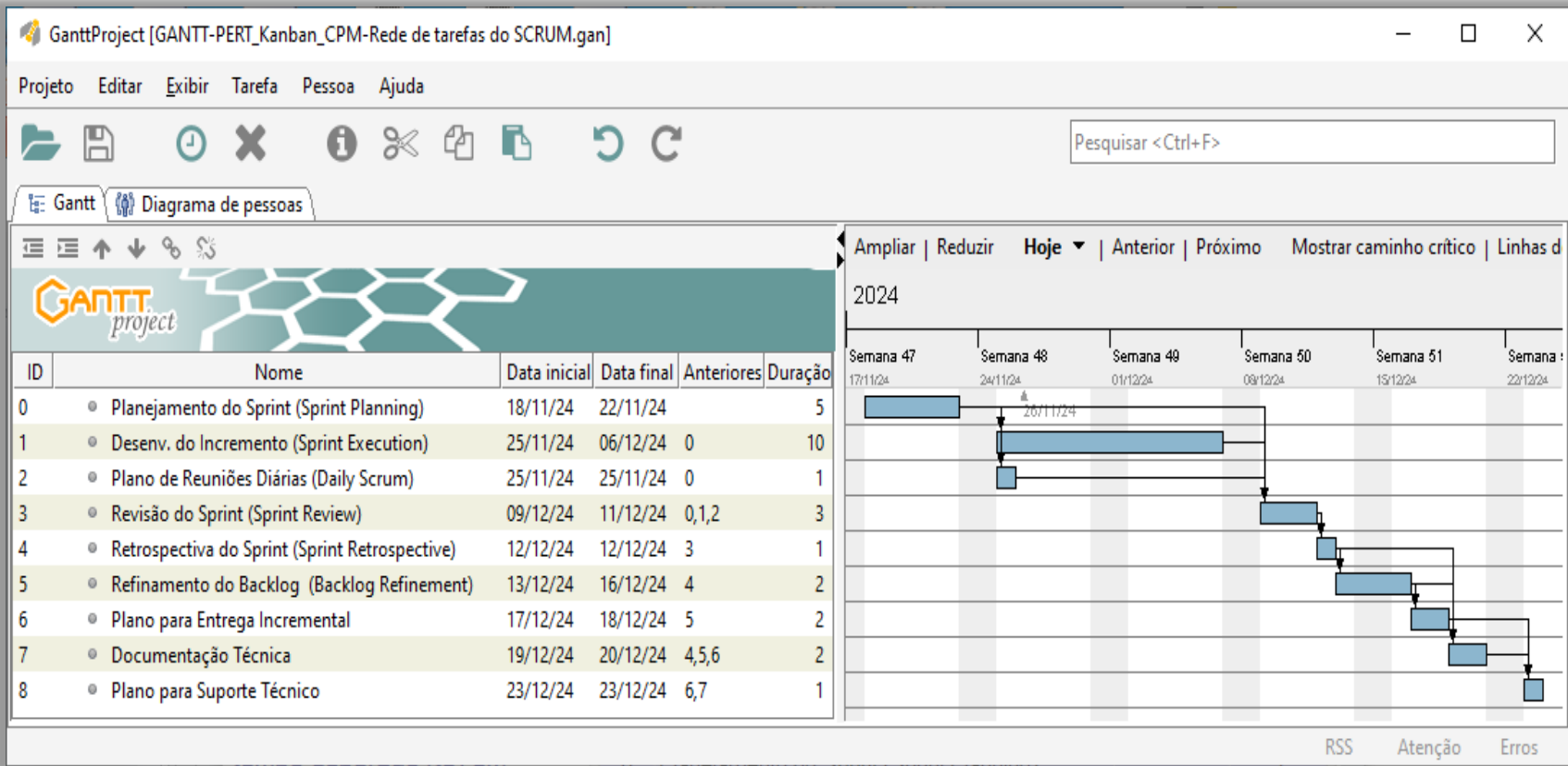
- Gestão de projetos – GanttProject (gratuito), com funções similares ao Project da Microsoft®; Trello, quadros e cartões e Teams da Microsoft® comunicação e compartilhamento.
- Modelagem – Construção de diagramas estruturados e orientados a objetos como Astah, BizAgi, Draw.io, Freemind e Visio da Microsoft®.
- IDE – Ambiente de desenvolvimento integrado para desenvolvimento de aplicações como ASP.NET, Django (Python), Eclipse, NetBeans e Xamarin (mobile, C#).

Ferramenta de Gestão de Projetos: GanttProject

- Com o GanttProject são obtidos cronogramas, gráfico de análise do caminho crítico da rede de tarefas (*kanban*) e diagrama de recursos (ou pessoas), entre outras aplicações. Esses gráficos e análises fornecem resultados da automação dos métodos PERT/CPM, matriz de responsabilidades (MR) e uso da metodologia ágil kanban (rede de tarefas).

Tela do software GanttProject com o cronograma gerado.

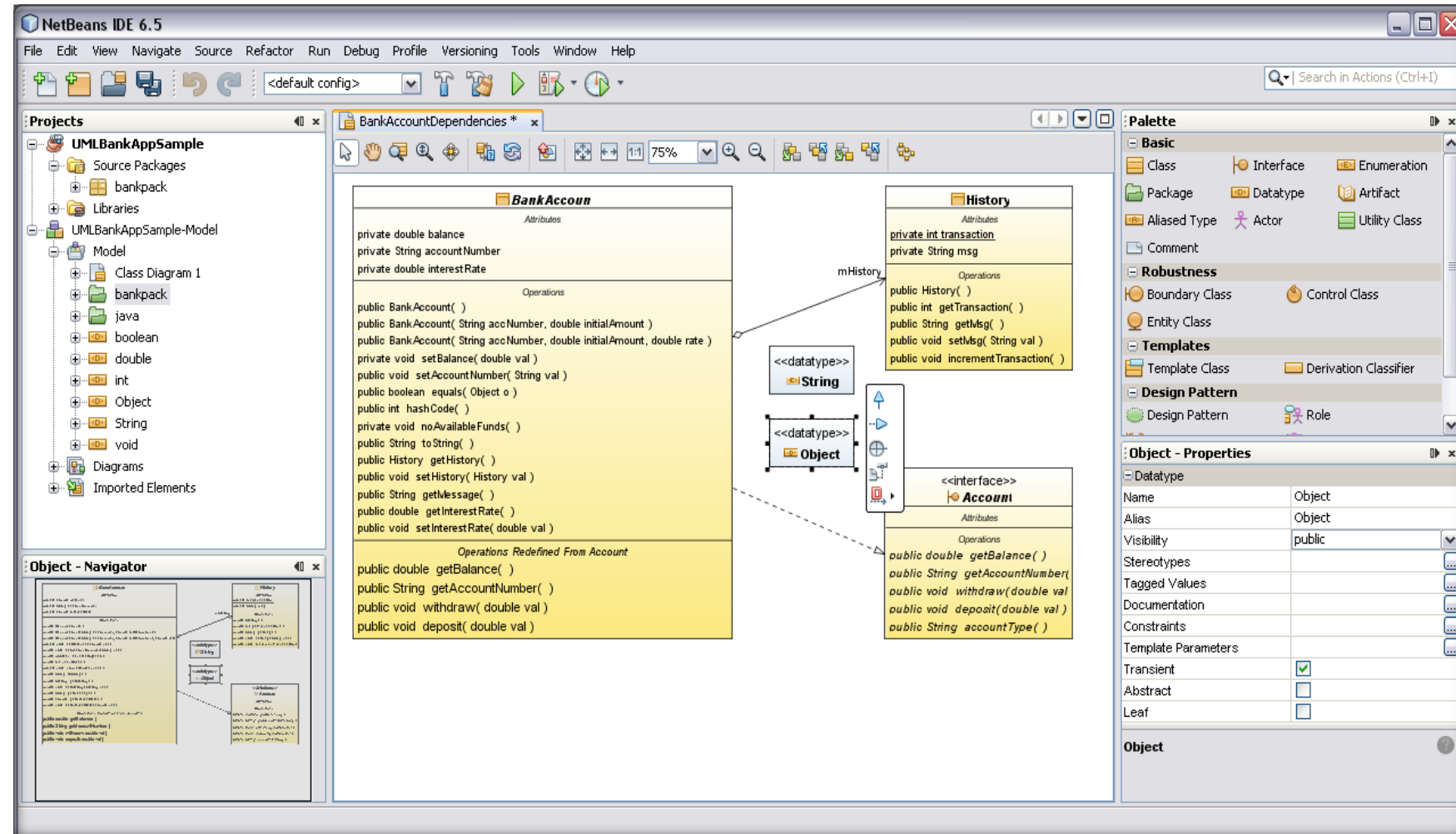
Fonte: Moreno (2025).



Ferramenta IDE: Framework Netbeans

- Funciona no domínio da aplicação. Usa basicamente um conjunto de classes e interfaces decompostas em quadros com seus respectivos códigos.

Ambiente de trabalho na codificação e modelagem de software com o framework de programação NetBeans.



Interatividade

As camadas da engenharia de software são: Qualidade (Q), Processos (P), Métodos (M) e Ferramentas (F). A lista abaixo corresponde a vários produtos e recursos disponíveis para desenvolver software:

Astah	CMMI	Eclipse	ISO 9126
ISO 12207	Modelo Cascata	Modelo Incremental	MPS.BR
Project da Microsoft®	Scrum	Visual Studio	XP

Associe respectivamente o produto ou recurso às camadas da engenharia de software:

- a) Q = CMMI; P = Modelo Incremental; M = Project da Microsoft; e F = Eclipse.
- b) Q = ISO 9126; P = CMMI; M = Visual Studio; e F = Eclipse.
- c) Q = ISO 9126; P = Modelo Incremental; M = Scrum; e F = Astah.
- d) Q = ISO 12207; P = Modelo Expiral; M = CMMI; e F = Visual Studio.
- e) Q = ISO 12207; P = Visual Studio; M = XP; e F = MPS.BR.

Resposta

As camadas da engenharia de software são: Qualidade (Q), Processos (P), Métodos (M) e Ferramentas (F). A lista abaixo corresponde a vários produtos e recursos disponíveis para desenvolver software:

Astah	CMMI	Eclipse	ISO 9126
ISO 12207	Modelo Cascata	Modelo Incremental	MPS.BR
Project da Microsoft®	Scrum	Visual Studio	XP

Associe respectivamente o produto ou recurso às camadas da engenharia de software:

- a) Q = CMMI; P = Modelo Incremental; M = Project da Microsoft; e F = Eclipse.
- b) Q = ISO 9126; P = CMMI; M = Visual Studio; e F = Eclipse.
- c) Q = ISO 9126; P = Modelo Incremental; M = Scrum; e F = Astah.
- d) Q = ISO 12207; P = Modelo Expiral; M = CMMI; e F = Visual Studio.
- e) Q = ISO 12207; P = Visual Studio; M = XP; e F = MPS.BR.

Joint Application Development (JAD)

- O Desenvolvimento de Aplicação Conjunta (JAD) é uma metodologia criada pela IBM em 1977, que tem por finalidade reunir uma equipe de usuários, stakeholders e desenvolvedores com objetivos de alcançar acordos baseados na compreensão das operações do processo.

Perfis dos principais participantes das sessões JAD:

- Facilitador (ou Líder da sessão) – Responsável pelo controle da sessão JAD.
- Executivo patrocinador – Fornece diretrizes das principais metas e objetivos do negócio.
- Gerência funcional – Representante do usuário que descreve as operações do negócio.
- Gerente de Projeto – Alinhando os recursos do projeto para produzir o software.
 - Gerente de Projeto de infraestrutura de TI – Atua junto à equipe de desenvolvimento.
 - Analistas de sistemas (e/ou Desenvolvedores) – São aqueles responsáveis pelas implementações.
 - Secretário (Escriba ou Documentador) – Utiliza ferramentas e técnicas documentais e de modelagem para os registros.

Gerenciamento da equipe de desenvolvimento

- Para promover um ambiente colaborativo e adaptável que atenda às necessidades de stakeholders e equipe do projeto, a metodologia sugerida baseia-se na Matriz de Responsabilidades (MR) do PMBOK® (Project Management Body of Knowledge).
- Também conhecida como MR RACI, ajuda a definir claramente papéis e responsabilidades de cada participante da equipe de projeto e atende a todo e qualquer modelo de equipe.

Principais aplicações da MR na engenharia de software

- Definição clara dos papéis e responsabilidades.
- Gestão de recursos.
- Integração com os métodos PERT/CPM.

Análise de dados e construção da matriz de responsabilidades

- Os stakeholders são todos os interessados no sistema de software, e em escala na hierarquia de decisão estão: cliente, representante do usuário, autor do processo, gerente de projeto, gerente do sistema, programador, administrador do banco de dados (DBA – DataBase Administrator).
- O conceito dos termos stakeholders e desenvolvedores é por livre e bom senso. Pode-se considerar “todos” como stakeholders ou, dependendo da organização, separar os stakeholders como aqueles interessados apenas no negócio.
- A figura mostra a distribuição hierárquica dos stakeholders.

STAKEHOLDER					
Cliente	Gerente de Projeto	Gerente de Sistemas	Analista de Sistemas	Program.	DBA

MR RACI – PMBOK®

- A matriz de responsabilidades (MR) do PMBOK® define claramente os papéis e responsabilidades dos participantes da equipe de desenvolvimento.
- A matriz é um arranjo de vários elementos num quadro retangular que comporta colunas e linhas.
- Colunas: 1ª – Identificação do registro (ID); 2ª – Tarefa (ou atividade); 3ª – Stakeholder – composição de várias colunas correspondentes aos envolvidos no projeto.
- Linhas: 1ª – Cabeçalho da matriz e as demais linhas são os registros de ID, Tarefas e células com os atributos RACI (R – Responsável pela tarefa; A – É quem Aprova (valida) a tarefa; C – Consultado (da suporte a R); I – Informado (é quem participa para questionar ou se manter atualizado no projeto)).

Uma boa distribuição dos atributos RACI permite não só atribuir funções específicas para os desenvolvedores, como também dimensionar melhor a carga de serviços e o custo associado a esse serviço.

MR RACI – PMBOK®: Modelo de MR com os atributos RACI

ID	Tarefa	Cliente	Gerente Projeto	Gerente Sistema	Analista Sistema	Program.	DBA
0	Objetivo do negócio	R	C	C	I	--	--
1	Requisitos do negócio	A	R	C	C	C	C
2	Análise de viabilidade	C	A	R	C	--	--
3	Modelagem dos casos de uso (RU)	A	C	R	C	--	--
4	Definição dos requisitos não funcionais (RNF)	A	R	C	C	--	--
5	Arquitetura da lógica de processamento (RF)	--	--	A	R	C	C
6	Arquitetura da informação (classes)	--	--	A	C	C	R
7	Arquitetura da infraestrutura de TI (RS)	C	A	R	C	I	I

Figura: Matriz de responsabilidades para a fase inicial do projeto de software. Fonte: Moreno (2025).

- Regras práticas para designação dos atributos RACI:
- R – Responsável: Um R apenas para cada tarefa.
 - A – Aprova: Valida a tarefa, não é necessário para todas as tarefas.
 - C – Consultado: Pelo menos um deve dar suporte à tarefa de R.
 - I – Informado: Participa da tarefa sem muita influência.

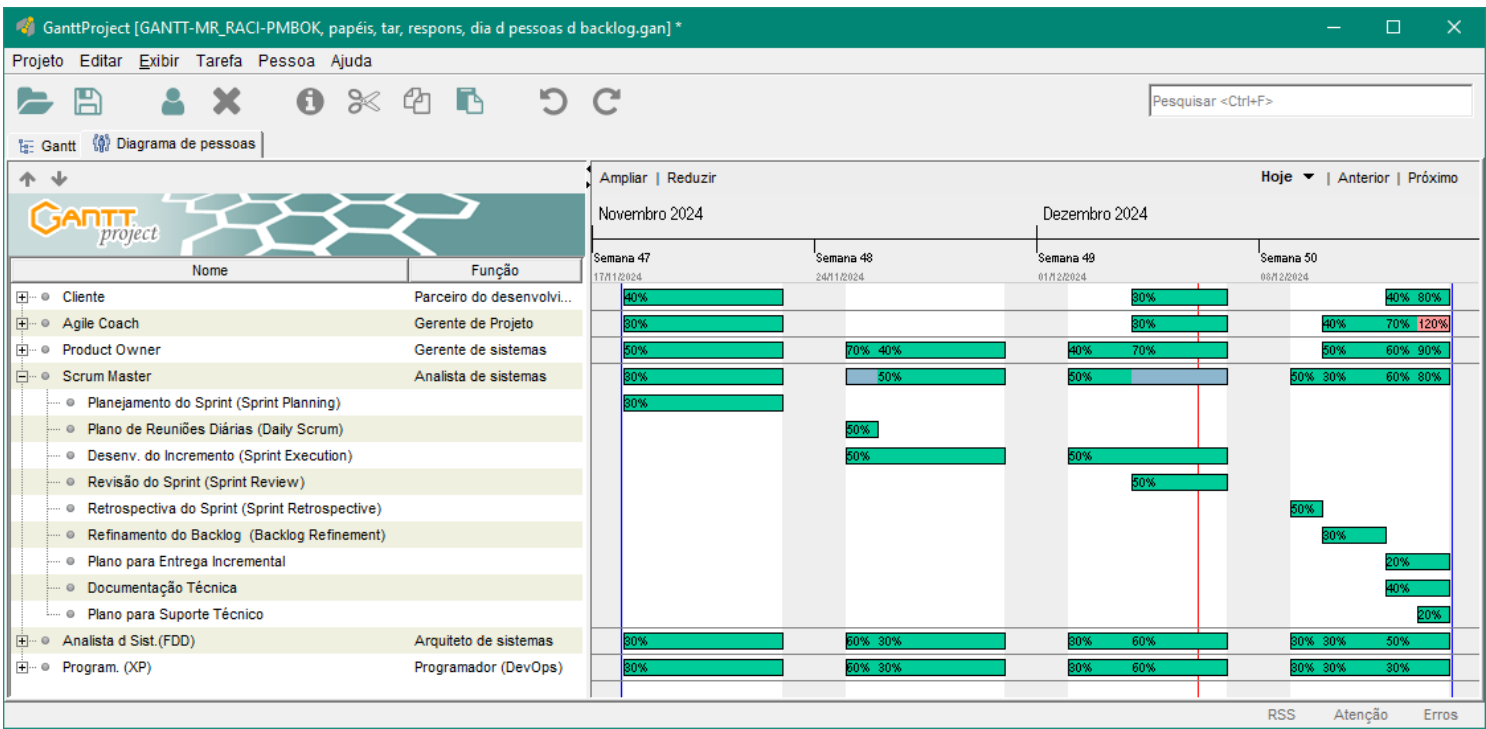
MR RACI – PMBOK®: Implementação com GanttProject

ID	Tarefa	Cliente	Gerente Projeto	Gerente Sistema	Analista Sistema	Program.	DBA
0	Objetivo do negócio	R	C	C	I	--	--
1	Requisitos do negócio	A	R	C	C	C	C
2	Análise de viabilidade	C	A	R	C	--	--
3	Modelagem dos casos de uso (RU)	A	C	R	C	--	--
4	Definição dos requisitos não funcionais (RNF)	A	R	C	C	--	--
5	Arquitetura da lógica de processamento (RF)	--	--	A	R	C	C
6	Arquitetura da informação (classes)	--	--	A	C	C	R
7	Arquitetura da infraestrutura de TI (RS)	C	A	R	C	I	I

Tabela construída em editor de textos.

Gráfico de pessoas obtido pelo GanttProject.

Fonte: Moreno (2025).



Método de análise da carga de serviços para cada stakeholder

- Esta análise é feita com valores estimados, apenas para dimensionar a distribuição das cargas de serviços.
- Neste modelo são atribuídas pontuações em %, por exemplo, que poderão ser convertidas em custos ou horas.

Tarefa	Stakeholders					
	Cliente	Agile Coach	Product Owner	Scrum Master	Analista Sist. (FDD)	Program. (XP)
Planejamento do Sprint (Sprint Planning)	A	C	R	C	C	C
Plano de Reuniões Diárias (Daily Scrum)	--	A	C	R	C	C
Desenv. do Incremento (Sprint Execution)	--	--	A	R	C	C
Revisão do Sprint (Sprint Review)	C	C	C	R	C	C
Retrospectiva do Sprint (Sprint Retrospec)	--	--	--	R	C	C
Refinamento do Backlog (Backlog Refine)	--	A	R	C	C	C
Plano para Entrega Incremental	A	R	C	I	--	--
Documentação Técnica	--	I	C	A	R	C
Plano para Suporte Técnico	A	R	C	I	--	--
SERVIÇO/PARTICIPANTE(%)	150	250	290	310	230	210
SERVIÇO/PARTICIPANTE (1/2 Período)	3	5	5,8	6,2	4,6	4,2
SERVIÇO/PARTICIPANTE (H.h: período 4h)	12h	12h	23h12	25h36	18h24	16h48

Atributos da matriz RACI com pontuações:

Fonte: Moreno (2025).

- R – Responsável (50%).
- A – Quem aprova (40%).
- C – Consultado (suporte) (30%).
- I – Informado (participa) (20%).

Interatividade

Analise os dados da matriz e responda, respectivamente, qual dos participantes é mais cobrado pelas atividades do projeto e o participante que mais valida as atividades do projeto.

ID	Tarefa	Cliente	Gerente de Projeto	Gerente de Sistema	Analista de Sistema	Program.	DBA
0	Objetivo do negócio	R	C	C	I	--	--
1	Requisitos do negócio	A	R	C	C	C	C
2	Análise de viabilidade	C	A	R	C	--	--
3	Modelagem dos casos de uso (RU)	A	C	R	C	--	--
4	Definição dos requisitos não funcionais (RNF)	A	R	C	C	--	--
5	Arquitetura da lógica de processamento (RF)	--	--	A	R	C	C
6	Arquitetura da informação (classes)	--	--	A	C	C	R
7	Arquitetura da infraestrutura de TI (RS)	C	A	R	C	I	I

- a) Analista de Sistema e Gerente de Sistema.
- b) Gerente de Projeto e Analista de Sistema.
- c) Gerente de Projeto e Gerente de Sistema.
- d) Gerente de Sistema e Cliente.
- e) Gerente de Sistema e Gerente de Projeto.

Resposta

Analise os dados da matriz e responda, respectivamente, qual dos participantes é mais cobrado pelas atividades do projeto e o participante que mais valida as atividades do projeto.

ID	Tarefa	Cliente	Gerente de Projeto	Gerente de Sistema	Analista de Sistema	Program.	DBA
0	Objetivo do negócio	R	C	C	I	--	--
1	Requisitos do negócio	A	R	C	C	C	C
2	Análise de viabilidade	C	A	R	C	--	--
3	Modelagem dos casos de uso (RU)	A	C	R	C	--	--
4	Definição dos requisitos não funcionais (RNF)	A	R	C	C	--	--
5	Arquitetura da lógica de processamento (RF)	--	--	A	R	C	C
6	Arquitetura da informação (classes)	--	--	A	C	C	R
7	Arquitetura da infraestrutura de TI (RS)	C	A	R	C	I	I

- a) Analista de Sistema e Gerente de Sistema.
- b) Gerente de Projeto e Analista de Sistema.
- c) Gerente de Projeto e Gerente de Sistema.
- d) Gerente de Sistema e Cliente.
- e) Gerente de Sistema e Gerente de Projeto.

Introdução ao processo unificado

- O desenvolvimento de software exige métodos organizados, flexíveis e voltados à entrega de produtos com qualidade e alinhados às necessidades do cliente.
- O Processo Unificado é um framework robusto, que combina práticas iterativas e incrementais com foco em arquitetura bem definida, gerenciamento de riscos e melhoria contínua.
- Sua estrutura permite que os projetos sejam desenvolvidos de forma controlada, divididos em fases bem definidas, o que facilita a adaptação a mudanças e a identificação de problemas desde as etapas iniciais.



Fonte: ClipArt.

Processo Unificado

O Processo Unificado incorpora diversas tecnologias para garantir que o desenvolvimento de software seja eficiente, estruturado e adaptável. Alguns exemplos dessas tecnologias são:

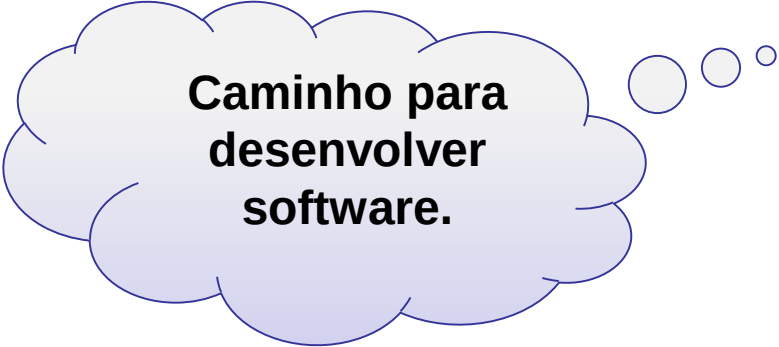
- O uso de modelagem orientada a objetos, baseada na UML.
- Foco na arquitetura de software, definida por casos de uso.
- O processo unificado valoriza práticas como engenharia de requisitos, automação de testes, integração contínua e ferramentas CASE (Computer-Aided Software Engineering), para a modelagem, codificação, versionamento e gerenciamento de projetos.
- O Processo Unificado busca manter um equilíbrio entre controle, documentação técnica e flexibilidade, adaptando-se a diferentes tecnologias e ambientes de desenvolvimento.

Neste capítulo serão abordados:

- Rational unified process (RUP): Fases e Workflow
- Extensões do RUP
- Métodos, ferramentas e técnicas (MF&T): Definição de papéis e responsabilidades para atividades do RUP

Processo Unificado – Ramificações

- Sua versão mais conhecida é o RUP (Rational Unified Process), do qual derivam diversas variações adaptadas a diferentes contextos.



**Caminho para
desenvolver
software.**

Ramificações do processo unificado:

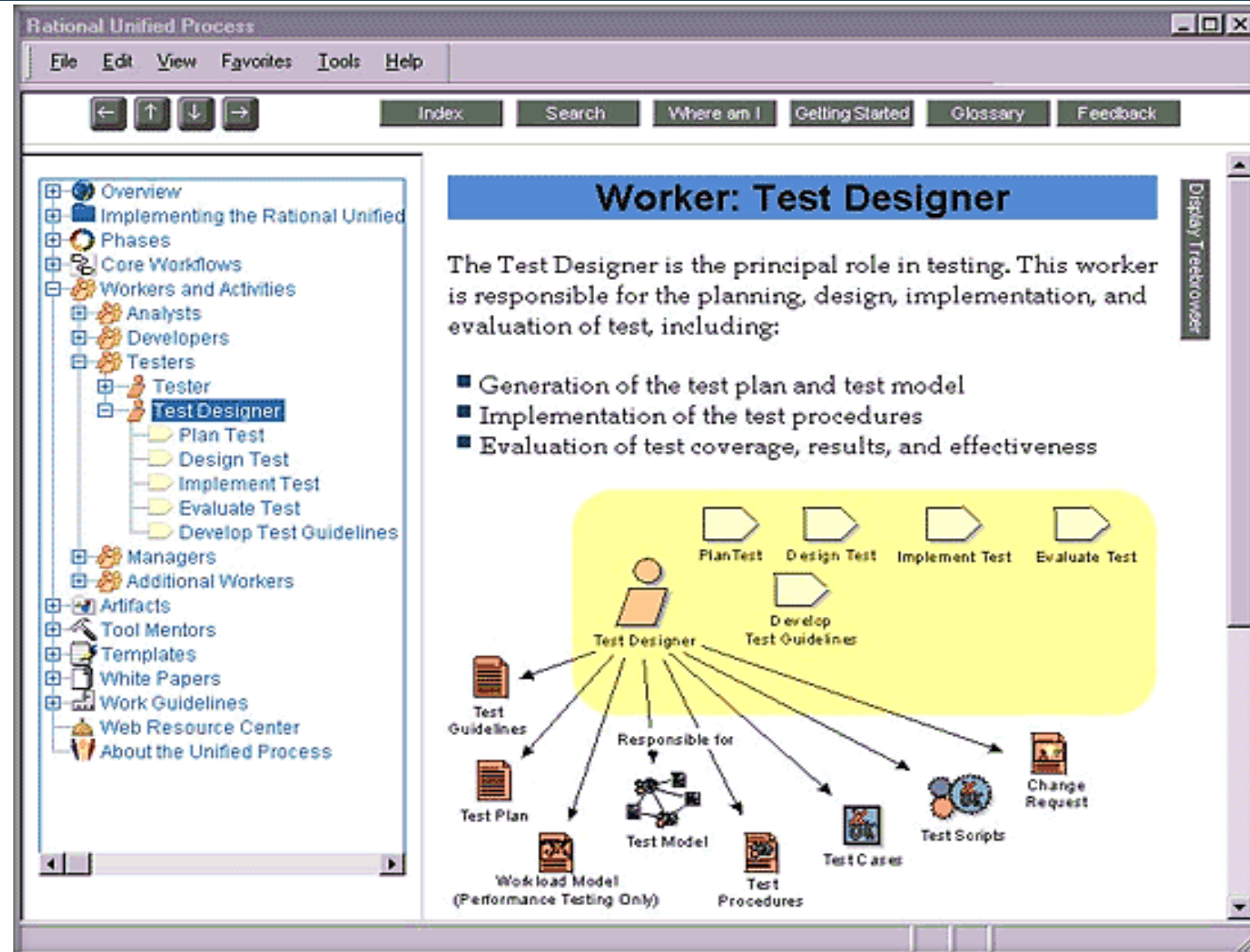
- RUP – Fornece um conjunto de diretrizes, ferramentas e modelos para apoiar o desenvolvimento de software.
- Praxix – Framework com base nos princípios do processo unificado com foco em sistemas embarcados e de tempo real.
- ICONIX – Abordagem ágil que simplifica o processo unificado.
- Processo Unificado Ágil (AUP) – Mantém principais fases do RUP, porém com iterações mais curtas e flexíveis.
- Enterprise Unified Process (EUP) – Extensão do RUP que incorpora as fases de produção e desativação do software.
- Open Unified Process (OpenUP) – processo unificado de código aberto, projetado para ser adaptado a diferentes tipos de projetos e equipes.

Rational Unified Process (RUP) – Fases e workflow

O RUP é um processo proprietário criado pela Rational Software Corporation, representado pela IBM®, com o nome IRUP (IBM Rational Unified Process).

Principais características:

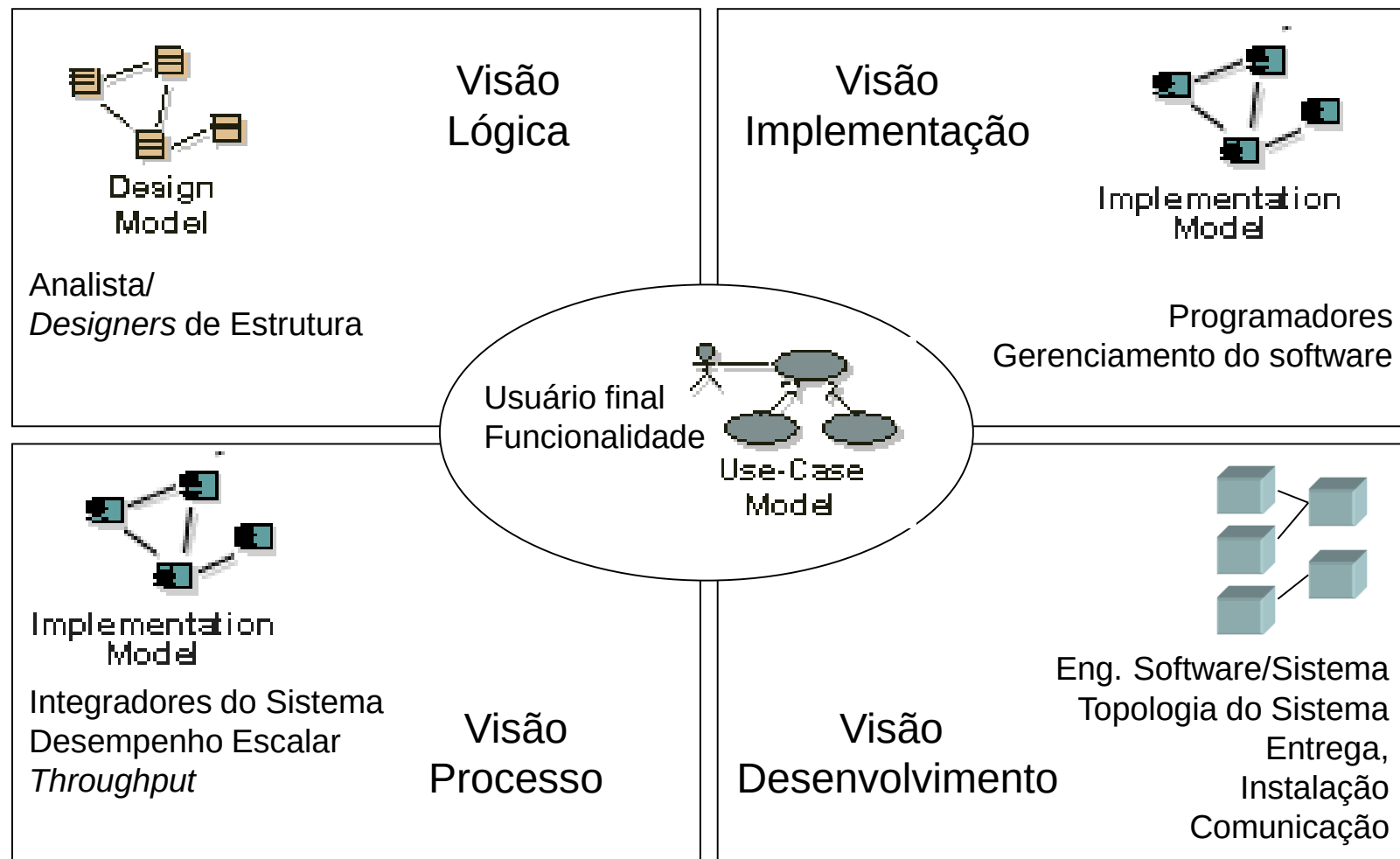
- Modelagem UML.
- Ferramentas de software e atualizações distribuídas pela web.



RUP – Orientado por casos de uso

A partir do caso de uso são criadas diversas visões sustentadas por diagramas UML, para compor a funcionalidade:

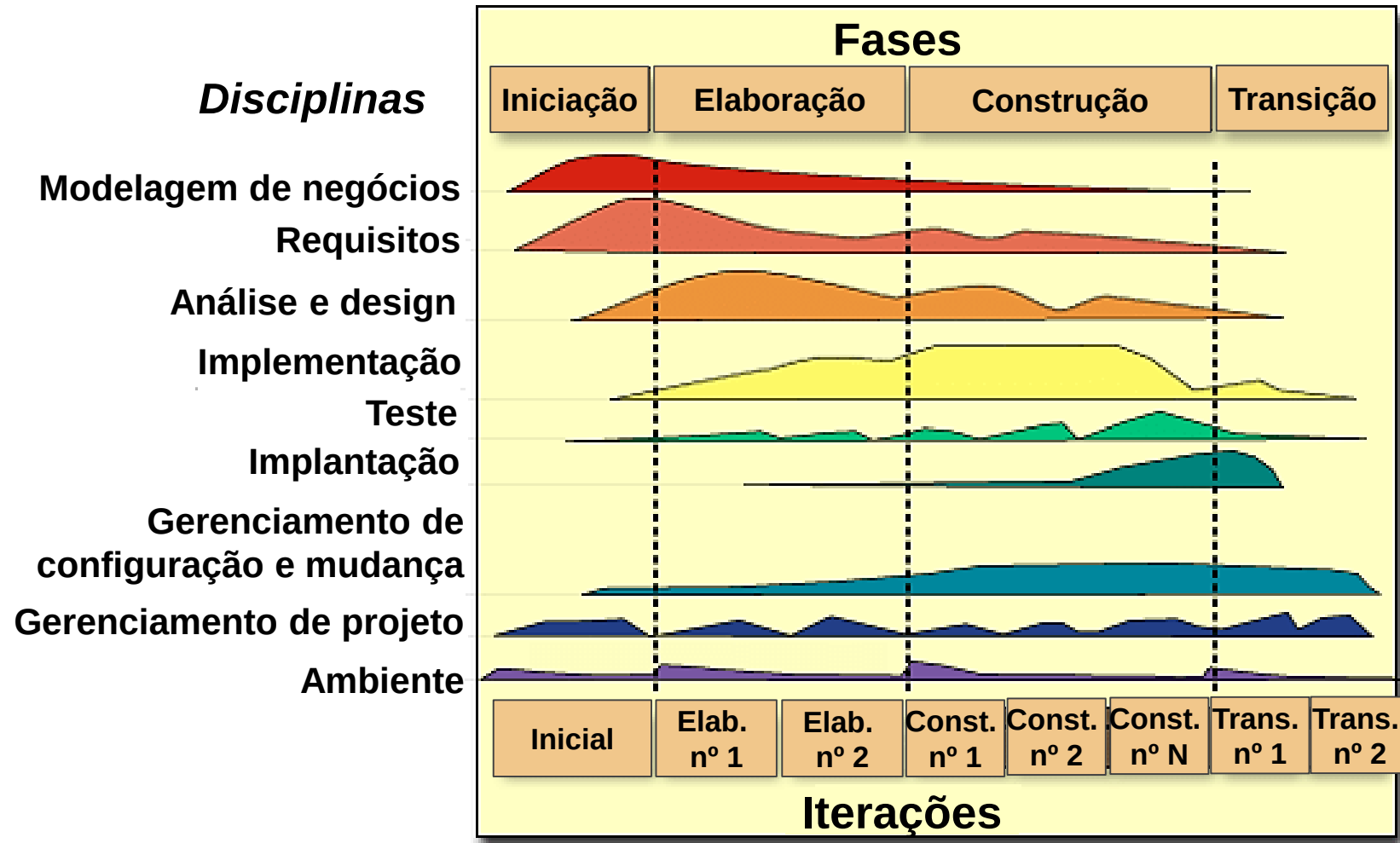
- Visão da Lógica
- Visão do Processo
- Visão do Desenvolvimento
- Visão da Implementação



Fonte: Krunchten (2000).

Framework do RUP

- Na horizontal são representadas as fases, que formam a estrutura dinâmica. São os períodos para a conclusão da fase.
- Na vertical são representadas as disciplinas (workflows), que formam a estrutura estática. São as atividades do desenvolvimento.



Estrutura Dinâmica

- Fase 1: Iniciação/Concepção (Inception/Conception) – Definir o escopo e as fronteiras do sistema; elaborar casos de uso (atores e requisitos) e o business case, e ao término da fase, obter validação dos stakeholders e desenvolvedores.
- Fase 2: Elaboração (Elaboration) – Obter estabilidade da arquitetura e requisitos; mitigar riscos; produzir protótipos para validação do projeto (tempo, custo); revisar business case; ao término, demonstrar que o projeto é viável (tempo, custo) e que o pode ser cancelado ou reformulado se falhar neste marco.
- Fase 3: Construção (Construction) – Implementação e testes das funcionalidades; desenvolver de forma iterativa e incremental, minimizar custos e entregar versões utilizáveis junto com a documentação. A equipe decide se o sistema de software pode ser implantado.
 - Fase 4: Transição / Implantação (Transition) – Entrega e implantação do software; fazer testes de integração; treinar usuários e responsáveis e equipe de manutenção. Obter satisfação dos clientes, gastos e custos dentro do aceitável.

Estrutura Estática – Elementos primários

Cada fase do ciclo de desenvolvimento resulta numa nova geração do produto; cada fase divide-se em iterações e a estrutura estática utiliza quatro elementos primários:

1. Trabalhadores (workers) – São os perfis e competências para a realização de atividades.

2. Atividades – São tarefas ou pacotes de tarefas entregues a trabalhadores individuais. São as atribuições de papéis e responsabilidades.

Papel	Atividade
Gerente de projetos	Planejamento das iterações
Gerente de sistemas	Especificação e modelagem
Analista de sistemas	Determina casos de uso e atores
Analista de testes de desempenho	Executa testes de desempenho

3. Artefatos – São os inputs e outputs das atividades.

4. Workflows – São os fluxos de relacionamento que agrupam atividades relacionadas.

Estrutura Estática – Workflow do RUP

- O workflow do RUP é apresentado em dois grupos compostos por seis disciplinas de trabalho e três disciplinas gerenciais, descritas a seguir:

Disciplinas de trabalho (6):

- Modelagem de Negócio (Business Modeling)
- Requisitos (Requirements)
- Análise e Projeto (Analysis and Design)
- Implementação (Implementation)
- Teste (Test)
- Utilização (Deployment)

Disciplinas gerenciais (3):

- Gerenciamento de Configuração
- Gerenciamento de Projeto
- Ambiente

Interatividade

O RUP é orientado por casos de uso. Qual das alternativas apresenta corretamente uma vantagem na orientação por casos de uso?

- a) Centraliza o desenvolvimento do código, reduzindo a importância do usuário.
- b) Elimina a necessidade de testes automatizados porque o caso de uso valida o software.
- c) Facilita o alinhamento entre requisitos e desenvolvimento.
- d) Permite o desenvolvimento sem a definição prévia de requisitos.
- e) Substitui a modelagem de design por fluxos de dados automáticos.

Resposta

O RUP é orientado por casos de uso. Qual das alternativas apresenta corretamente uma vantagem na orientação por casos de uso?

- a) Centraliza o desenvolvimento do código, reduzindo a importância do usuário.
- b) Elimina a necessidade de testes automatizados porque o caso de uso valida o software.
- c) **Facilita o alinhamento entre requisitos e desenvolvimento.**
- d) Permite o desenvolvimento sem a definição prévia de requisitos.
- e) Substitui a modelagem de design por fluxos de dados automáticos.

Extensões do RUP

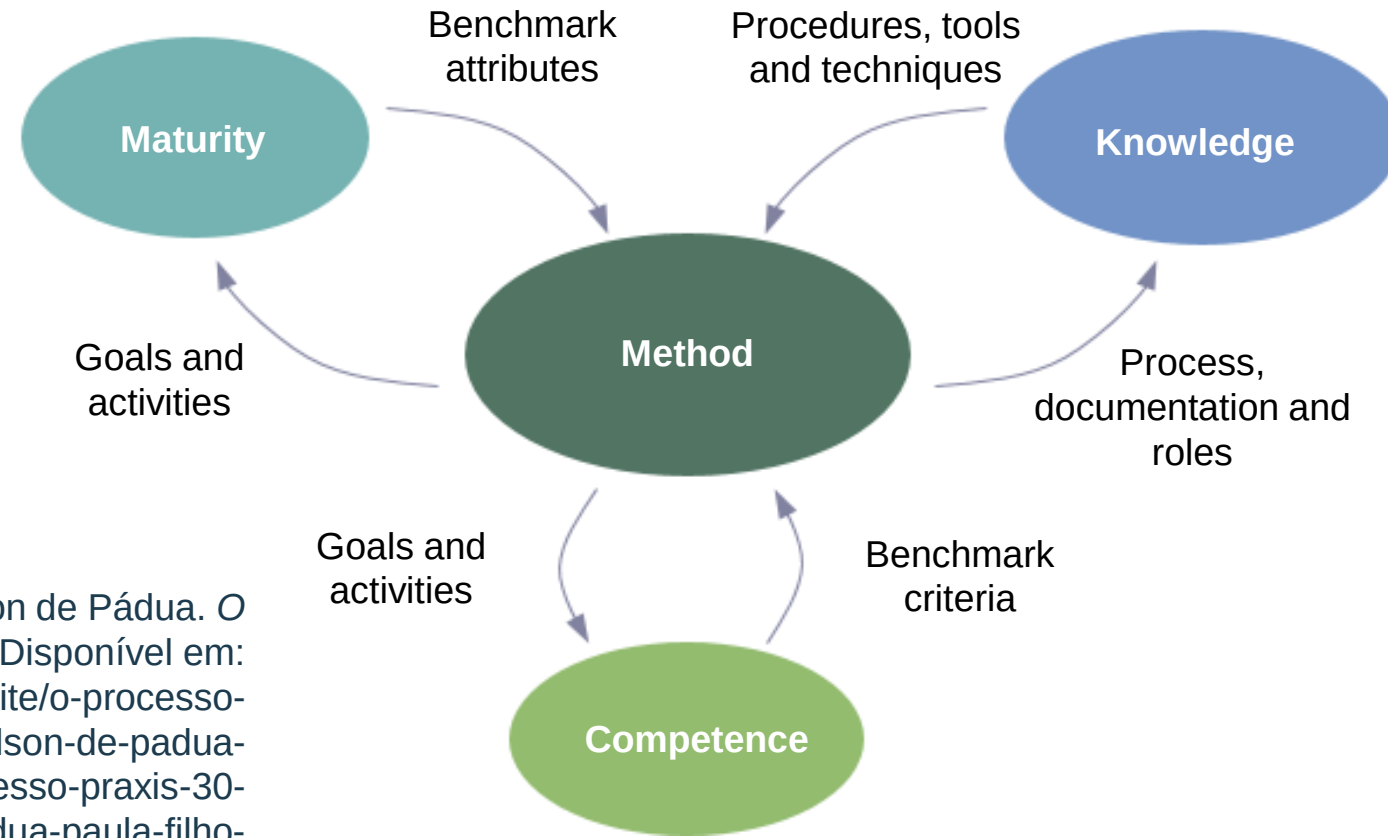
- As extensões do RUP (Rational Unified Process) são variações, adaptações ou complementações desse processo de desenvolvimento de software, para atender a contextos específicos de projeto, empresa ou cultura organizacional.

Principais extensões do RUP:

- Praxis – O Praxis real-time é um método formal para o desenvolvimento de sistemas críticos como: sistemas embarcados, aplicações em tempo real, sistemas aeroespaciais, médicos ou militares.
- ICONIX – Abordagem ágil como alternativa mais simples do RUP, usado para: Modelagem UML, Engenharia de Requisitos, modelagem de domínio e casos de uso.
- Enterprise Unified Process (EUP) – Ciclo de vida completo do sistema integrando à gestão de TI, além do tradicional foco do RUP. Adiciona fases e disciplinas como gerenciamento do portfólio, operações, suporte, aposentadoria do sistema.
- Open Unified Process (OpenUP) – Versão simplificada de código aberto do processo unificado, incorporando princípio ágil, projetado para ser adaptado a diferentes tipos de projetos e equipes.

Praxis

- O modelo Praxis tem base no modelo de ciclo de vida espiral, com as mesmas raízes no processo unificado e estrutura diferente de disciplinas.
- O modelo Praxis, igualmente ao RUP, é dirigido por casos de uso, centrado na arquitetura, iterativo e incremental.



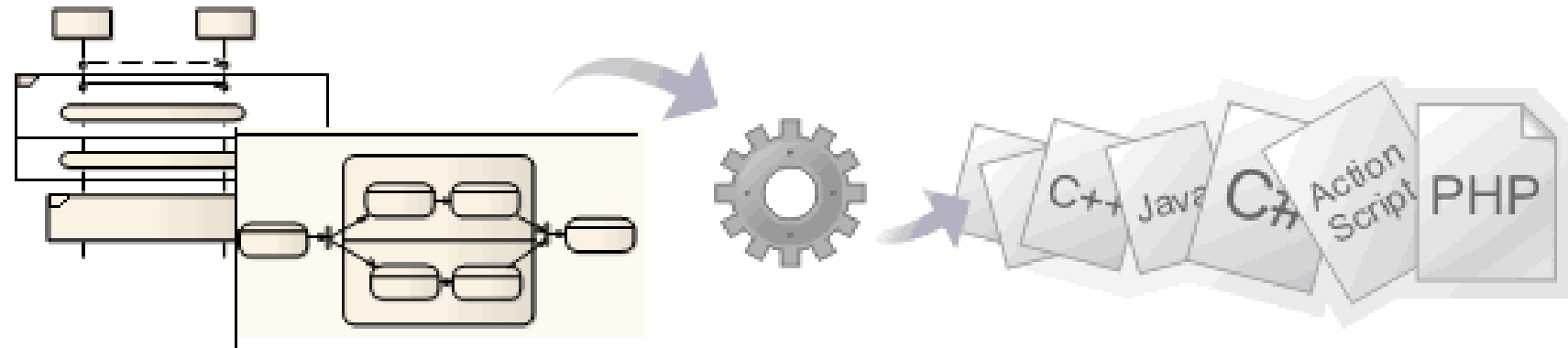
PAULA FILHO, Wilson de Pádua. *O Processo Praxis 3.0*. Disponível em: <https://vdocuments.site/o-processo-praxis-30-2008-wilson-de-padua-paula-filho-o-processo-praxis-30-wilson-de-padua-paula-filho-professor-aposentado-dcc-icex-ufmg-diretor.html>, 2008. Acesso em: 02 ago. 2025.

PRAXIS. Praxis Framework Limited, 2019. Fonte: <https://www.praxisframework.org/en/method>. Acesso em: 02 ago. 2025.

ICONIX

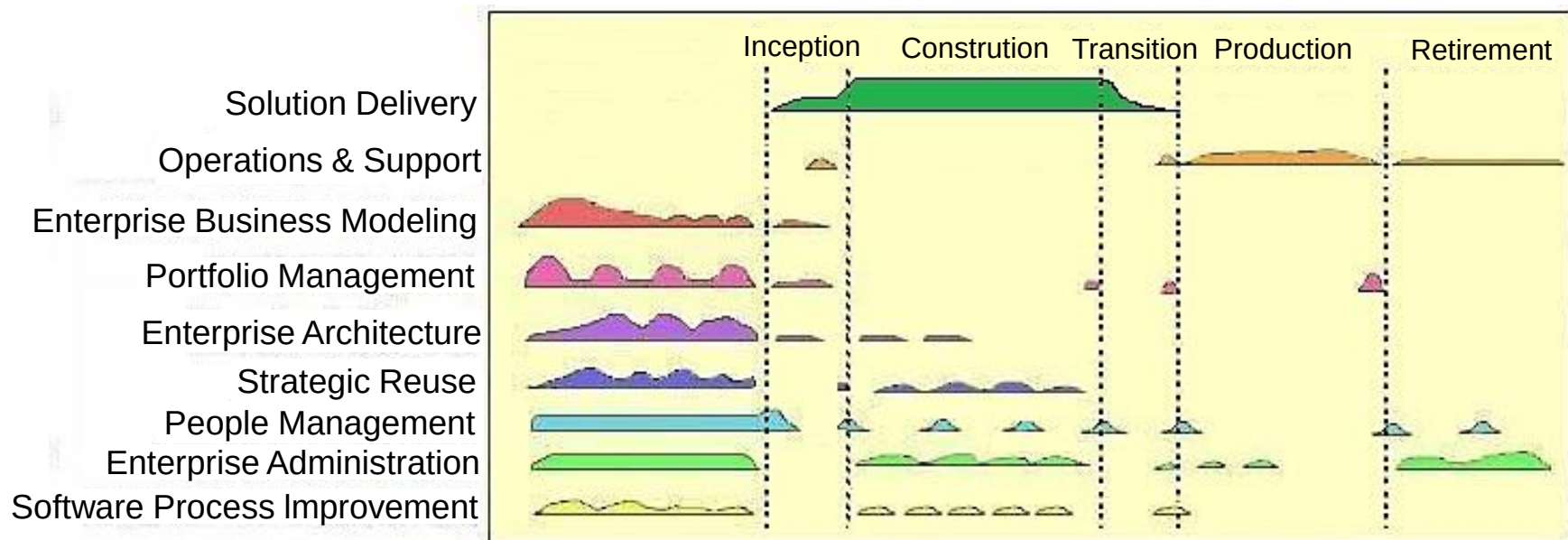
- A metodologia ICONIX antecede tanto o Processo Unificado Racional (RUP), a Programação Extrema (XP), quanto as práticas de desenvolvimento ágil.
- Os modelos comportamentais apresentados na ilustração são construídos com a ferramenta Enterprise Architect.
- Esta ferramenta “compila” modelos a partir de diagramas específicos, como máquina de estado, sequência, atividades e outros modelos da UML, para gerar código na linguagem descritiva do sistema de software. As linguagens suportadas incluem C(OO), C++, C#, Java, VB.Net, VHDL, Verilog e SystemC.

Figura – Modelos comportamentais baseados em diagramas da UML são gerados pelo Enterprise Architect na automatização da codificação



Enterprise Unified Process (EUP)

- O EUP é uma extensão do RUP que visa tratar de todo o ciclo de vida do software, não se limitando apenas ao desenvolvimento, incorporando no ciclo de vida as fases Production (Produção) e Retirement (Desativação), como pode ser observado no gráfico da figura.



Disponível em: https://enterpriseunifiedprocess.com/wp-content/uploads/2023/02/eup_lifecycle_2013.jpg. Acesso em: 20 mar. 2025.

Fonte: Ambler (2023).

Open Unified Process (OpenUP)

- O OpenUP é uma metodologia de desenvolvimento de software que faz parte do Eclipse Process Framework (EPF), que é uma versão mais leve e ágil do RUP.
- Basicamente, possui o mesmo ciclo de vida estruturado do RUP, com as quatro fases principais: Concepção, Elaboração, Construção e Transição, mantendo suas características essenciais, porém mais simplificado.

Existem dois objetivos do Eclipse Process Framework Project (Eclipse (2025)):

1. Possui uma estrutura de ferramentas para engenharia de processos de software, para criação de métodos, processos, gestão de bibliotecas, configuração e publicação do processo.
2. Fornecer conteúdo de processo extensível para uma variedade de processos de desenvolvimento e gerenciamento de software que suportem desenvolvimento iterativo, ágil e incremental, e aplicável a um amplo conjunto de plataformas e aplicativos de desenvolvimento.

Métodos, ferramentas e técnicas (MF&T) – Definição de papéis e responsabilidades para atividades do RUP

- Perfis profissionais:
Cliente (CL), Usuário (US),
Analista de Negócios (AN),
Gerente de Projeto (GP),
Gerente de Sistema (GS),
Analista de Sistemas (AS),
Programação (PR),
Tester (TE) e Administrador
do Banco de Dados (DB).

id	Disciplinas	Stakeholders e Desenvolvedores								
		CL	US	AN	GP	GS	AS	PR	TE	DB
1	Modelagem de Negócios	A	C	R	C	C	I	--	--	I
2	Requisitos	C	C	A	C	R	C	I	--	I
3	Análise e Design	C	C	C	A	R	C	I	I	I
4	Implementação	I	I	C	I	C	C	R	C	C
5	Teste	I	C	I	I	A	C	C	R	C
6	Implantação	C	A	C	R	C	C	C	C	C
7	Ger. de Config. e Mudança	--	--	--	C	R	C	C	C	C
8	Ger. de Projeto	A	C	C	R	C	C	C	C	C
9	Ambiente	C	C	C	C	A	R	C	I	I
Fases		Iniciação		Elaboração		Construção			Transição	

Atributos da matriz RACI: R – Responsável; A – quem Aprova; C – Consultado (suporte); I – Informado (participa).

Quadro – Matriz de distribuição de papéis e responsabilidades para o processo unificado

Fonte: Moreno (2025).

Interatividade

Das extensões do RUP, o OpenUP é uma das que mais se destaca por incorporar princípio ágil. Qual das alternativas aponta corretamente uma de suas características estruturais?

- a) É um framework fechado com foco em ferramentas estruturais para compor módulos do sistema.
- b) É um framework que utiliza uma integração do modelo em cascata com o RUP, não prevendo customização de fases.
- c) Estrutura baseada nos requisitos limitando o conteúdo técnico e ágil por não abordar práticas tradicionais do desenvolvimento.
 - d) Oferece um ciclo de vida sequencial, voltado para grandes equipes.
 - e) Estrutura simplificada, iterativa e de código aberto, com suporte a ferramentas do processo de software.

Resposta

Das extensões do RUP, o OpenUP é uma das que mais se destaca por incorporar princípio ágil. Qual das alternativas aponta corretamente uma de suas características estruturais?

- a) É um framework fechado com foco em ferramentas estruturais para compor módulos do sistema.
- b) É um framework que utiliza uma integração do modelo em cascata com o RUP, não prevendo customização de fases.
- c) Estrutura baseada nos requisitos limitando o conteúdo técnico e ágil por não abordar práticas tradicionais do desenvolvimento.
- d) Oferece um ciclo de vida sequencial, voltado para grandes equipes.
- e) Estrutura simplificada, iterativa e de código aberto, com suporte a ferramentas do processo de software.

Referências

- AMBLER, Scott W. *Enterprise Unified Process (EUP): Estratégias para o Enterprise Agile*. Ambsoft Inc: Enterprise Unified Process, 2023. Disponível em: <https://enterpriseunifiedprocess.com>. Acesso em: 02 ago. 2025.
- ECLIPSE. Eclipse Process Framework Project. © Eclipse Foundation AISBL. Disponível em: <https://projects.eclipse.org/projects/technology.epf#>. Acesso em: 20 mar. 2025.
- KRUCHTEN, Philippe. *The Rational Unified Process: An Introduction*. 2a ed. USA: Addison-Wesley, 2000.
- PRESSMAN, Roger S.; MAXIM, Bruce R. *Engenharia de Software: uma abordagem profissional*. Porto Alegre: AMGH, 2021. (1348 p.)
 - SPARX. *ICONIX – Guia do usuário do arquiteto corporativo*. Sparx Systems. Disponível em https://sparxsystems.com/enterprise_architect_user_guide/15.2/model_domains/iconix_process.html. Acesso em: 02 ago. 2025.

ATÉ A PRÓXIMA!